Autonomous Systems Lab

Path Control - Python Simulation

Abstract

This document discusses the theoretical background and practical implementation of path control for a two-wheel mobile robot. We cover wheel odometry, sensor fusion (wheel odometry + IMU), path-tracking algorithms (pure pursuit, Stanley), trapezoidal velocity profiling, and PID control. Laboratory simulations and exercises are included to illustrate how these concepts translate to real-world scenarios.

1. Introduction

Path control is fundamental to autonomous mobile robotics, ensuring that a robot follows a predefined trajectory while accounting for noise and uncertainties in sensor measurements. This paper provides both the theoretical underpinnings (odometry, sensor fusion, motion profiles) and practical implementation details (PID, code structures, and lab simulations) for a small Zumo-like robot.

1.1 Roadmap

- **Section 2** covers wheel odometry basics, including the derivations for estimating position and orientation.
- **Section 3** introduces sensor fusion between IMU and odometry for smooth and more stable estimate
- **Section 4** introduces path-control algorithms, comparing pure pursuit, Stanley, and point-to-point (P2P) strategies.
- **Section 5** explores trapezoidal velocity profiles and how they can be combined with path control for smooth, jerk-free motion.
- **Section 6** discusses PID control and how it is integrated into motor speed regulation and heading control.
- **Section 7 outlines the lab simulation instructions (Python simulator) and recommended parameter-tuning steps.**
- **Section 8** presents in-lab tasks for testing these ideas on a physical platform.

2. Wheel Odometry and Kinematic Model

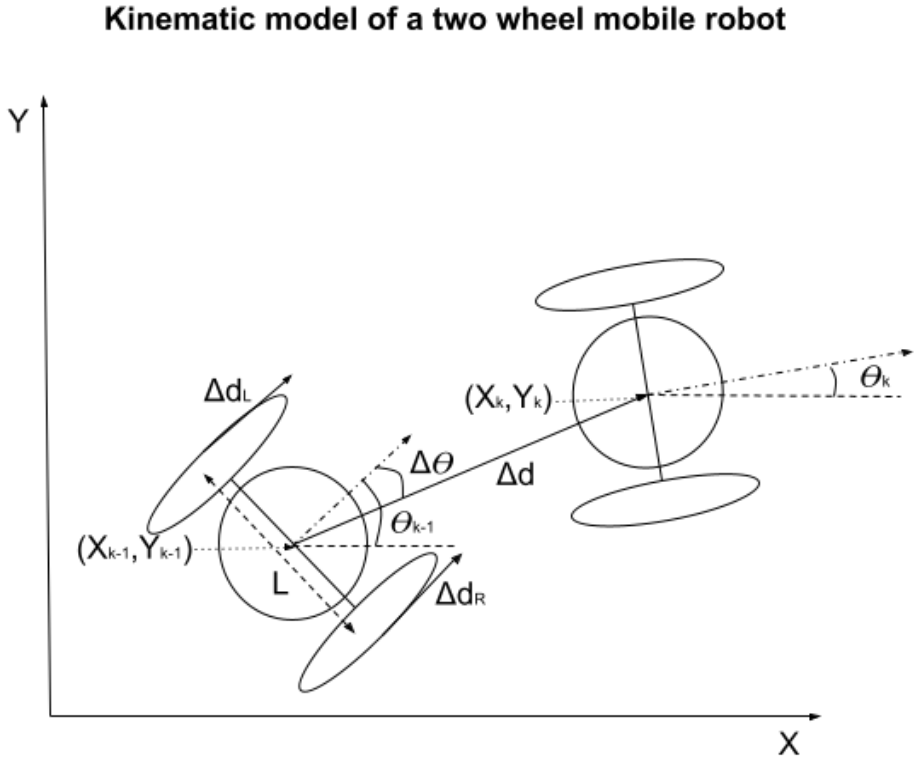
2.1 Basic Wheel Odometry

Wheel odometry estimates robot motion using rotation measurements from wheel encoders.

- **Key Components**
  - **Wheel Encoders:** Measure wheel rotation in discrete ticks per revolution.
  - **Robot Geometry:** Wheel diameter and the baseline (distance between the two wheels) are critical for calculating forward motion and turns.
- **Advantages**
  - Straightforward, purely on-board sensing (no external infrastructure).
  - Relatively easy to implement with off-the-shelf encoder hardware.
- **Limitations**
  - Susceptible to slipping and skidding (assumes stable contact with the ground).
  - Errors accumulate over time without external correction.

2.2 Kinematic Model

A common two-wheel mobile robot model (with differential drive) can be derived from left- and right-wheel travel distances:



- $\Delta d = \frac{\Delta d_R + \Delta d_L}{2}$

- $\Delta\Theta = \frac{\Delta d_R - \Delta d_L}{L}$
- $\Delta d_{(L,R)} = \boldsymbol{\pi} D_{(L,R)} \frac{EncoderPulses}{EncoderPulsesPerRevolution}$
- $X_k = X_{k-1} + \Delta d \cdot cos(\Theta_{k-1} + \Delta\Theta/2)$
- $Y_k = Y_{k-1} + \Delta d \cdot sin(\Theta_{k-1} + \Delta\Theta/2)$
- $\Theta_k = \Theta_{k-1} + \Delta\Theta$

where:

- Δd_R and Δd_L are distances traveled by the left and right wheels.
- L is the wheelbase (distance between left and right wheels).
- X_k, Y_k and Θ_k are the new position and orientation at timestep kkk.

3. Sensor Fusion: Wheel Odometry + IMU

3.1 IMU Components

An Inertial Measurement Unit (IMU) typically includes:

- **Accelerometer** (measures linear acceleration),
- **Gyroscope** (measures angular velocity),
- **Magnetometer** (measures orientation relative to Earth's magnetic field).

3.2 Complementary Filter for Fusion

Because gyros drift over time and accelerometers introduce noise, combining IMU and wheel-encoder data yields a more stable estimate. A complementary filter is a simple yet effective approach:

Gyro Orientation Change:

$$\Delta\theta_{gyro} = \omega \cdot \square\square$$

Wheel Odometry Orientation Change:

$$\Delta\theta_{odo} = \frac{\Delta d_R - \Delta d_L}{L}$$

Complementary filter:

$$\Theta_k = \square \cdot (\square_{k-1} + \Delta\theta_{gyro}) + (1 - \square) \cdot (\Theta_{k-1} + \Delta\theta_{odo})$$

Where:

- d: is the distance travelled
- L: is the distance between wheels
- ω: is angular velocity from the gyro,
- α: is a tuning parameter (0 to 1).

Note: More sophisticated methods (e.g., Extended Kalman Filter) can also be employed, especially if the environment is prone to noise, drift, or large disturbances.

4. Path Control

Accurate path control ensures the robot follows a desired path, often represented by discrete waypoints or a continuous curve. Two widely known approaches are **Pure Pursuit** and **Stanley**.

4.1 Pure Pursuit

1. **Core Idea**: The robot looks ahead to a “target point” on the path and steers to intercept it.
2. **Key Parameter**: **Lookahead Distance**, which determines how far ahead the robot aims along the path.
3. **Advantages**:
    - Simple to implement, easy to tune.
    - Effective on relatively smooth paths.
4. **Disadvantages**:
    - Might overshoot tighter curves.
    - Performance depends heavily on the lookahead parameter.

4.2 Stanley Algorithm

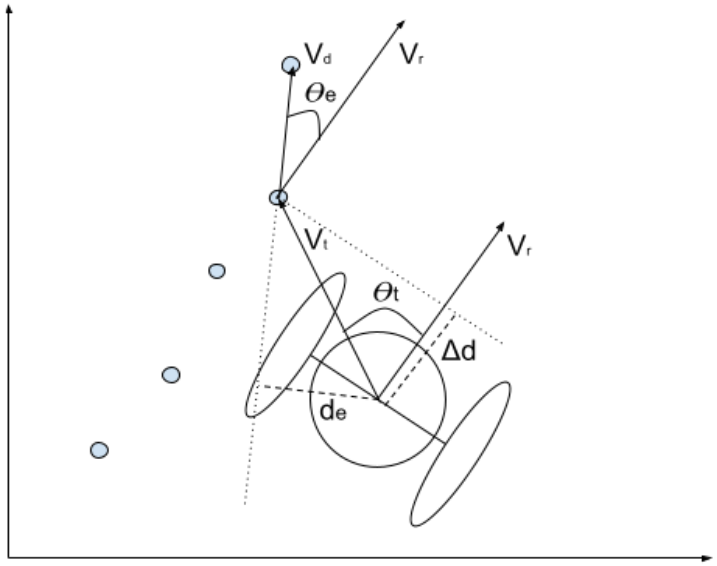
1. **Core Idea**: Uses cross-track error and heading error relative to the trajectory to compute a steering command.
2. **Advantages**:
    - Often better at handling sharper turns and “stickier” path tracking.
3. **Disadvantages**:
    - More complex, slightly higher computation.
    - Requires careful tuning of proportional gains for cross-track error and heading alignment.

4.3 Point-to-Point (P2P) Control

For simpler tasks, you can steer from your current position to a single desired point by calculating:

1. The bearing difference θ_t between the robot’s heading and the vector from the robot to the target.
2. A forward command proportional to distance to the target.
3. A turning command proportional to θ_t

Path control of a two wheel mobile robot



Cross product:

- $\underline{a} \times \underline{b} = \left||a|\right| \cdot \left||b|\right| \cdot sin(\Theta)\hat{n}$

Dot product

- $\underline{a} \cdot \underline{b} = \left||a|\right| \cdot \left||b|\right| \cdot cos(\Theta)$

Robot direction vector

- $V_r = [cos(\Theta)\,, sin(\Theta)]$

Point to Point (P2P) Control can be express by

- $\Theta_t = arccos(\frac{V_t \cdot V_r}{||V_t|| \cdot ||V_r||}) \cdot sign(V_r \times V_t)$

Trajectory Control can be expressed by

- $\Theta_e = arccos(\frac{V_d \cdot V_r}{||V_d|| \cdot ||V_r||}) \cdot sign(V_r \times V_d)$
- $d_e = \frac{V_t \times V_d}{||V_d||}$
- $\Delta d = \frac{V_t \cdot V_r}{||V_r||}$

5. Trapezoidal Velocity Profiles

5.1 Rationale

A trapezoidal velocity profile transitions smoothly through acceleration, constant velocity, and deceleration phases. This improves motion accuracy and reduces wear on motors/gears.

Key Phases

1. **Acceleration**: Increase velocity from 0 to V_{max} at a_{max}
2. **Constant Velocity**: Maintain V_{max}
3. **Deceleration**: Reduce velocity back to 0 by $-a_{max}$

5.2 Governing Equations

Using basic kinematics:

- $V_f^2 = V_0^2 + 2a\Delta s$

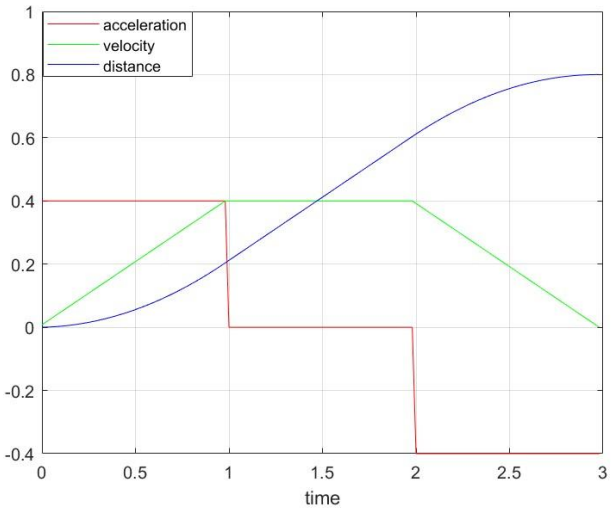
and for distance-based scheduling:

- $V_k = \sqrt{2a \cdot s}$

subject to constraints:

- $V_k \leq V_{max}$, $\frac{\Delta V_k}{\Delta t} \leq a_{max}$

An example of a desired motion profile, with max acceleration and speed limit.



6. PID Control

A **Proportional-Integral-Derivative (PID)** controller reduces error between a setpoint (desired speed or heading) and the current measured value.

$$u(t) = K_p \cdot e(t) + K_i \cdot \int e(t)dt + K_d \cdot \frac{de(t)}{dt}$$

where:

- $e(t)$ is the error,
- K_p, K_i, K_d are gains for proportional, integral, and derivative terms.

6.1 Example: PID with FIR Derivative Filter

```
float kp = 1.0; // Proportional gain
float ki = 0.1; // Integral gain
float kd = 0.2; // Derivative gain
float Ts = 0.01; // Sampling time
float integral = 0.0; // Integral accumulator
float prev_error = 0.0; // Error from previous iteration
float prev_derivative = 0.0; // Derivative from previous iteration
float derivative = 0.0; // Current derivative value
float alpha = 0.1; // Filter coefficient
float filtered_derivative = 0.0; // Filtered derivative value

float pid_control(float setpoint, float process_variable) {
    // Calculate error
    float error = setpoint - process_variable;

    // Calculate integral term
    integral += error * Ts;

    // Calculate derivative term with FIR filter
    derivative = (error - prev_error) / Ts;
    filtered_derivative = alpha * derivative + (1 - alpha) * prev_derivative;

    // Calculate control signal
    float control_signal = kp * error + ki * integral + kd * filtered_derivative;

    // Store previous error and derivative for next iteration
    prev_error = error;
    prev_derivative = filtered_derivative;

    return control_signal;
}
```

Tip: In real applications, consider clamping the integral term to avoid “windup” and manage noise in derivative calculations.

6.2 PI Wheel-Speed Controller Used in the Simulation (Python)

Each wheel has its own PI speed controller (zumo_sim/motor.py). The output is the motor PWM duty, limited to [-1, 1], with clamping anti-windup: the integral is frozen while the output is saturated.

```
class PIController:
    def __init__(self, kp, ki, limit=1.0):
        self.kp, self.ki, self.limit = kp, ki, limit
        self.integ = 0.0

    def update(self, error):
        pre = self.kp * error + self.integ             # PI output before saturation
        out = min(self.limit, max(-self.limit, pre))   # PWM duty limited to [-1, 1]
        saturated = pre != out
        if not (saturated and (pre > 0) == (error > 0)):
            self.integ += self.ki * error              # running sum, once per 10 ms sample
        return out
```

Note: the integral is a running sum of Ki·e, evaluated once per 10 ms sample. The sample time is not multiplied in. This is how the Simulink PID blocks (accumulation mode) and the robot firmware implement it, so Ki = 0.01 per sample is equivalent to a continuous-time Ki of 0.01 / 0.01 s = 1.0 s^{-1}.

7. Pre-Lab Activity (Python Simulation)

Overview & Objectives

Before the hardware lab session, you will explore a Python simulation of the two-wheel robot's motion and control. The simulator is a direct port of the four MATLAB/Simulink models (Simplified_OpenLoop, ZumoSimulatiom_P2P, ZumoSimulatiom_P2P_MultiPoint and ZumoSimulatiom_Path_Control), and its results agree with them to within a few millimetres. You can use either environment. MATLAB is not required. The goals of this pre-lab are:

1. **Familiarize yourself with the simulation environment** (the interactive GUI, the zumo_sim Python package, and the example scripts).
2. **Observe how changes to key control parameters** (e.g., PID gains, trapezoidal velocity limits, path-control gains) affect robot behavior.
3. **Compare different path control strategies** (point-to-point vs. path-following) to understand their strengths and limitations.
4. **Gain insight** into how simulation responses differ from real-world performance, laying the groundwork for your in-lab experiments.

1. Setting Up the Python Environment

The simulator is in the project GitHub folder ZumoPi_V02/Python_Simulation. It needs **Python 3.10 or newer** and the packages numpy, matplotlib and scipy. The GUI uses tkinter, which comes with the standard Python installer. The folder's README.md has detailed instructions for Windows, macOS and Linux.

- **Install Python** from python.org. On Windows, tick **Add python.exe to PATH** during installation.
- **Create a virtual environment** (a private copy of Python for this project). Install the packages and start the GUI. Run these commands in a terminal opened in the Python_Simulation folder:

```
python -m venv .venv                  # create the virtual environment (once)
.venv\Scripts\activate                # activate - Windows
source .venv/bin/activate             # activate - macOS / Linux
pip install -r requirements.txt       # numpy, scipy, matplotlib (once)
python gui.py                         # start the simulator
```

- **Check the installation (optional):** `python tools/validate_against_matlab.py` compares the Python simulation with results exported from the MATLAB models and prints the differences in mm.
- **Recommended editor:** VS Code with the Python extension. Use **Python: Select Interpreter** and choose .venv.

2. Simulation Roadmap

All four MATLAB simulations are available in the Python simulator. Select them with the **Controller** drop-down in the GUI, or with the mode argument of simulate():

Simulation	Purpose	Key Observations/Outcomes
Simplified open loop (Simplified_OpenLoop)	Robot motion where the motor commands come directly from the position error. There is no wheel-speed PI.	Notice the limitations of control without wheel-speed feedback.
Point to point – single point (ZumoSimulatiom_P2P)	Point-to-Point (P2P) control to a single target, with PI motor speed control.	See how the PI loop and the θ gain affect reaching a single target (overshoot, settling time).
Point to point – multi way-point (ZumoSimulatiom_P2P_MultiPoint)	Extends P2P to multiple sequential waypoints.	Observe transitions between waypoints and the overall path error.
Path control (ZumoSimulatiom_Path_Control)	Full path following, using heading and cross-track errors relative to the path segment.	Compare performance to P2P. Tune the path-control gains.

Each Simulink block has a Python counterpart. Open these files to see how the algorithms are implemented:

- `zumo_sim/controllers.py` – the Path Control and Point to Point MATLAB function blocks (PathController, P2PController). **This is the file to study and modify.**
- `zumo_sim/simulation.py` – the block diagram: rate limiter (trapezoidal profile), θ and de gains, wheel PI loops and odometry, executed every 10 ms.
- `zumo_sim/params.py` – InitZumo.m and all gains in one place (ZumoParams).
- `zumo_sim/motor.py` – DC motor, 75:1 gearbox, Hall encoder and the PI controller.
- `zumo_sim/odometry.py` – encoder position estimation (Section 2.2).

3. Key Parameters to Explore

The left panel of the GUI groups the parameters that influence the robot's behavior:

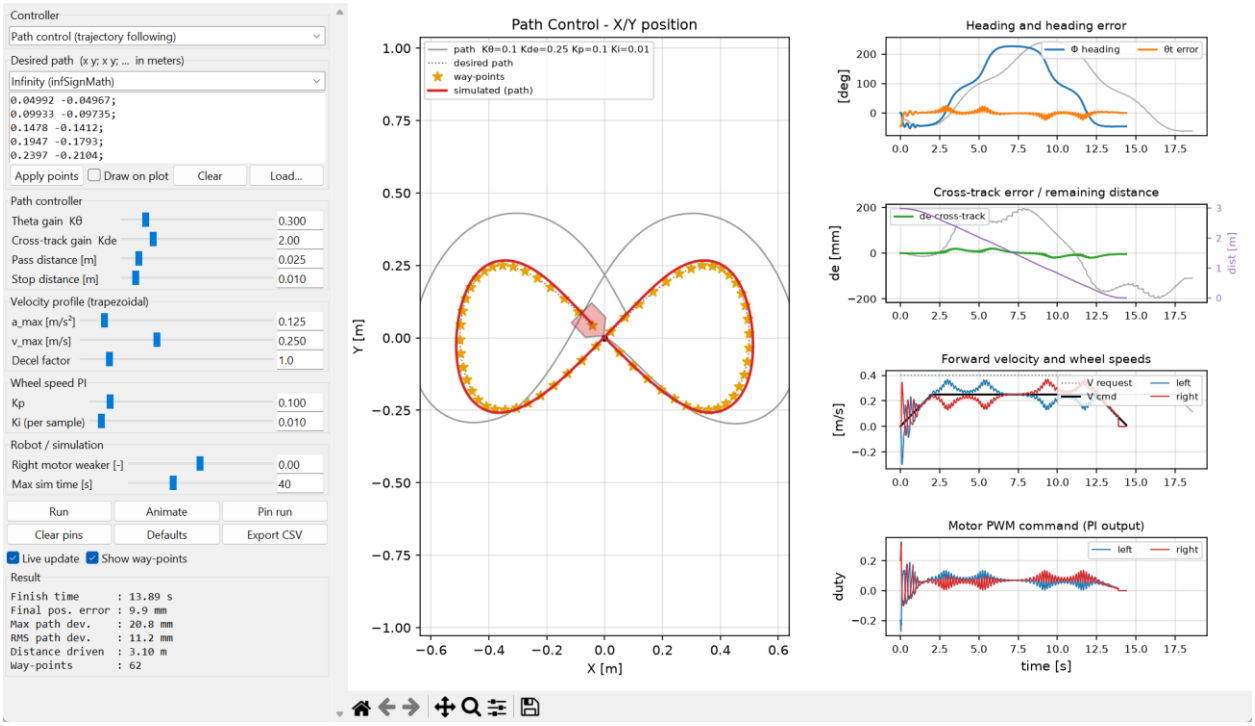
- **Motor PID Gains**: $K_p, K_i,$ for the motors' speed control.
- **Trapezoidal Profile Settings**: Acceleration a_{max}, maximum velocity V_{max}.

- **Path Control Gains**: For heading (θ gain) and cross-track error
- **Way-point logic:** pass distance (advance to the next point) and stop distance (stop at the last point).
- **Motor mismatch:** makes one motor weaker than the other. This is not in the MATLAB model, but it is a useful way to see how robust each controller is to real hardware differences.

Note: *Changing these parameters can increase or decrease overshoot, settling time, and steady-state error.*

4. The Simulator GUI

Start the GUI with `python gui.py`. The 2D path is on the left. The time responses (heading, cross-track error, velocities and PWM commands) are on the right.



- **Controller:** choose Path control, Point to point or Simplified open loop.
- **Desired path:** choose a preset, or type the points as x y; x y; ... (MATLAB style, in meters) and press Apply points. With **Draw on plot** ticked, left-click on the X-Y plot adds a point and right-click removes the last one. **Load...** reads a two-column text/CSV file.
- **Sliders:** every change re-runs the simulation automatically. You can also type an exact value in the box next to a slider and press Enter.
- **Pin run** keeps the current result on the plot (grey) so you can compare it with the next change. **Clear pins** removes them.

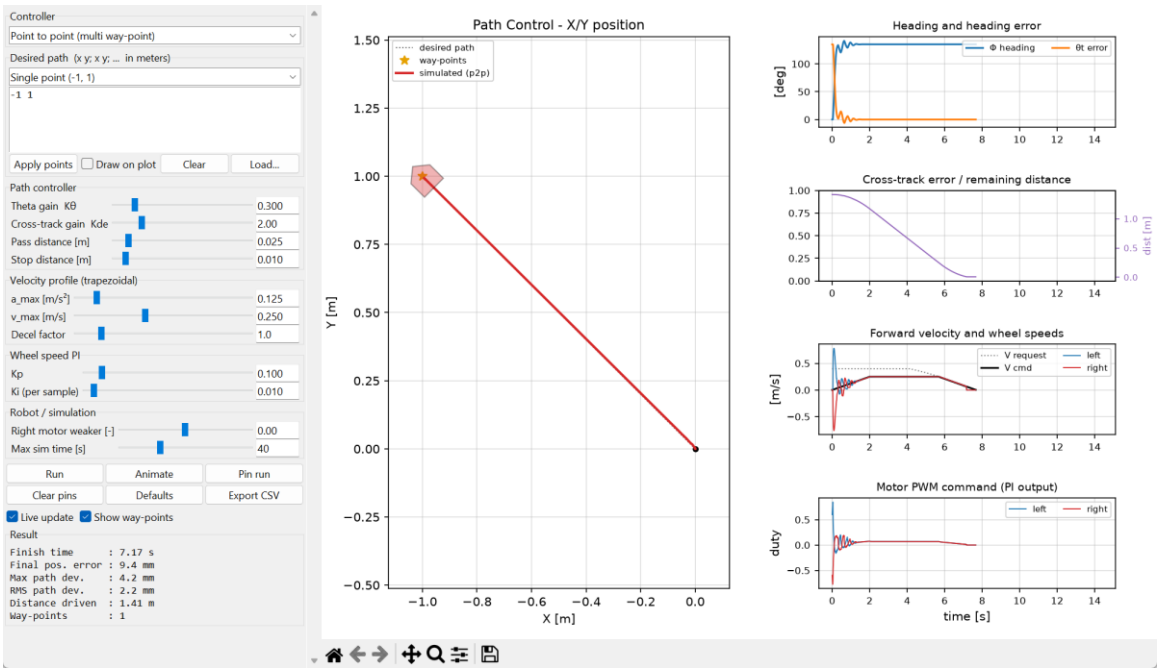
- **Animate** replays the motion. The circle marks the way-point the controller is currently tracking.
- **Result** panel: finish time, final position error, and maximum / RMS deviation from the desired path. Use these numbers to compare tuning experiments.
- **Export CSV** saves all signals of the current run. The toolbar under the plots zooms and saves the figure as an image.

5. Step-by-Step Instructions

1. **Setup**
   - Activate the virtual environment and run `python gui.py`.
   - Press **Defaults** to load the default parameters. These are the same values as InitZumo.m and the Simulink models.

2. **Simplified open loop** (Simplified_OpenLoop)
   - Choose the preset **Single point (-1, 1)** and select the controller **Simplified open loop**.
   - Observe the robot's trajectory, the wheel speeds and the PWM commands. Note how the absence of wheel-speed feedback affects the motion. Compare with the P2P result in the next step.

3. **Point to point – single point** (ZumoSimulatiom_P2P)



   - Select **Point to point** with the preset **Single point (-1, 1)**. You can also type -1 1 in the path box and press Apply points.
   - Inspect how the robot moves to a single target (x,y).
   - Adjust the **motor PID gains** for example, increase K_p , K_i or add more θ gain value, and observe the changes in the trajectory.

- Press **Pin run** before each change so the previous result stays on the plot for comparison.
- **Record** your observations in a short table (parameter changed → resulting behavior).

4. **Point to point – multi way-point** (ZumoSimulatiom_P2P_MultiPoint)
   - Keep **Point to point** selected and enter the waypoints `0 0.2; 0 0.4; 0 0.6; 0 0.8; 0 1` (or the preset Line 1 m).
   - Run the simulation and examine the time plots: heading, remaining distance, velocity profile and PWM commands.
   - **Try extending the path to 2 meters** (e.g., `0 0.5; 0 1.0; 0 1.5; 0 2.0`).
   - Observe how the robot transitions from one waypoint to the next and whether the trapezoidal velocity profile or PID gains cause any instability between points.

5. **Path control** (ZumoSimulatiom_Path_Control)
   - Select **Path control** and use the same points as in the previous step.
   - Pay special attention to the **heading error** (θ gain) and **cross-track error** (de gain).
   - Gradually increase or decrease these gains to see how the robot “tracks” a continuous path.
   - **Plot** the final path the robot follows and compare it to the desired path. Look for consistent offsets, oscillations, or overshoot around tight turns.
   - **Square path:** load the preset **Square 0.5 m** with the default gains. Does the robot complete the path? Use the way-point logic in PathController.update() (pass distance and the next-point-is-closer test) to explain what happens. Then find at least one fix: different gains, a different pass distance, or a change to the code. P2P mode completes this path. Why?

6. **Data Plotting and Analysis**
   - The GUI records and plots the robot's position, heading, errors, velocities and PWM commands. Save figures with the toolbar's save icon, or export the data of a run with Export CSV.
   - For scripted analysis, call the simulator from Python, for example `r = simulate(points, "path", ZumoParams(k_theta=0.3))` and then `plt.plot(r.x, r.y)`. The result also holds t, phi, theta_t, de, dist, v_cmd, wl/wr (wheel speeds) and ul/ur (PWM).
   - `examples/gain_sweep.py` sweeps one gain. It plots the trajectories and prints a table of finish time, final error and path deviation. This is a good starting point for the tuning deliverable.

7. **Concluding tasks**
   - Open `examples/plot_script.py` (the Python version of plotScript.m). It runs the P2P and Path control simulations on the variable **DesiredCoordinates** and plots both:

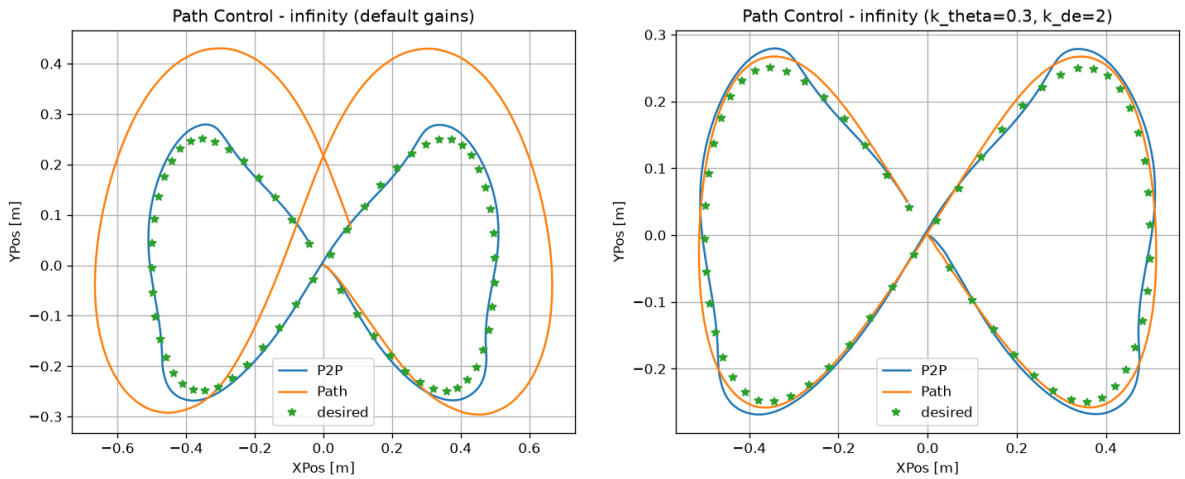
```
from zumo_sim import ZumoParams, simulate, paths

DesiredCoordinates = paths.line(2.0, 0.2)       # [0 0.2; 0 0.4; ... 0 2]
# DesiredCoordinates = paths.infinity()          # infinity sign (infSignMath.m)

Record_P2P  = simulate(DesiredCoordinates, "p2p",  ZumoParams(decel_factor=2.0, k_theta=0.1))
Record_Path = simulate(DesiredCoordinates, "path", ZumoParams(k_theta=0.1, k_de=0.25))

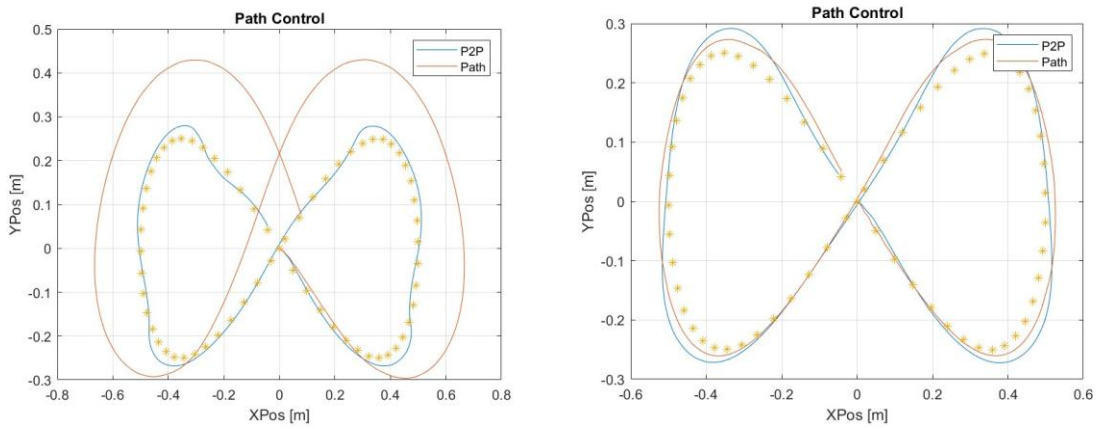
plt.plot(Record_P2P.x, Record_P2P.y, label="P2P")
plt.plot(Record_Path.x, Record_Path.y, label="Path")
```

- Adjust the algorithm's gains for optimal results without inducing jitter in the robot's angle. Plot the response and explain how each gain affects the system's behaviour.



- Define a path of your choosing (up to 1x1 metres) either by using discrete points or a continuous path from which 50-100 discrete points are derived. Run the simulation using the coordinates you have defined. Adjust the algorithm`s gains / parameters as needed for optimal performance and plot the response.

Example:



- **Optional:** `examples/custom_controller.py` is a template for writing your own controller. The example steers toward a look-ahead point (Pure Pursuit, Section 4.1). Compare its tracking with the original Path controller, and explain the difference.

6. Deliverables

By the end of the pre-lab, you should have:

1. **A short summary** (bullet points or a paragraph) of your observations for each simulation (OpenLoop, P2P, MultiPoint, PathControl).
2. **Plots or screenshots** illustrating at least one gain-tuning experiment.
3. **Written answers** (brief notes) to the guiding questions, focusing on how parameter variations changed the robot's simulated performance and why.

Conclusion

Completing this Pre-Lab Activity will deepen your understanding of basic control concepts—odometry, PID, trapezoidal velocity profiles, and path-tracking algorithms—before working hands-on with the physical robot.

8. In-Lab Activity

Overview & Objectives

This in-lab activity will allow you to:

- **Deploy and test your path-control algorithms** on an actual two-wheel mobile robot
- **Compare real-world performance** to the pre-lab simulation results, identifying similarities and differences due to factors like wheel slip, encoder noise, and imperfections in sensor measurements.
- **Refine your control gains** (motor PI, path-control parameters) to account for real dynamics.
- **Log and analyze** data collected from the robot to evaluate performance metrics such as cross-track error, oscillation. Compare the actual path to the desired path.

1. Running Simple Path Control

Note that the actual platform uses different units than the simulation. (The simulation uses meters, m/s. while the code on the platform implemented as millimetre, mm/s)

- Open Example **PathControl.ino** under WS_Zumo/Arduino/Examples/PathControl/
- Copy the project to your student folder.
- Make sure the path coordinates are se to:

```
// Define desired path (coordinates in mm).
float desiredPath[][2] = {
  {100, 0},
  {200, 100},
  {300, 100}
};
```

- Under **PathController.cpp** you will find the various path control settings. Make sure to start with the default values.

```
// Initialize control parameters.
dt_time  = 0.01f;           // 10 ms update interval
WHEELS_DISTANCE = 98.0f;    // mm between wheels
a_max = 200.0f;             // maximum acceleration (mm/s^2)
v_max = 100.0f;             // maximum forward velocity (mm/s)
Kp = 1.0f;                  // Velocity control Proportional gain
Ki = 0.1f;                  // Velocity control Integral gain
Kp_theta = 100.0f;          // Path control theta heading error gain
Kp_de = 1.0f;               // Path Control cross-track error gain
```

- Run the code. Power on the motors and press A button to start motion.
- Observe how well it reaches the marked target points:

2. Real time observer application

- Open Example **PathExample.py** under WS_Zumo/Python/Examples/
- Copy the project to your student folder.
- The example records the odometry messages to **recorded_messages.txt** In addition it will plot the X,Y position in 2D. In a separate plot it will present several motion parameters over time.
- Run the application open chrome or explorer navigate to your car ip at port 8500 (eg: http://192.168.0.120:8500/)
- View the position in real time during the motion.
- Plot the path from the **recorded_messages.txt** using Python (or MATLAB).
- Mark the path coordinates on the plot.

3. Tuning the path control gains, PI Velocity controller.

- Open **PathControl.ino** under your student folder.
- Review the Path control gains.
- Adjust the Path control gains. θ_e, d_e (**Kp_theta, Kp_de**) Change only one parameter at a time.
- Test what happens when one of the parameters is set to 0.
- Record several tuning experiments and create a plot in Python (or MATLAB) comparing the results.

4. Trapezoidal Profile Experiments

- Increase **v_max** or **a_max** in small increments to see if the robot can maintain stable control at higher speeds with higher acceleration.
- Observe if higher speeds lead to larger final errors, especially in turning maneuvers.

5. Full path control

- Implement the path from the prelab. Remember to scale the parameters accordingly.
- Record the results and compare them to the desired path.

6. Optional: Introduce Sensor Fusion

- Using the gyroscope from the IMU try implementing a complementary filter for better heading estimation.
- Compare the heading stability with and without gyro integration.

7. Deliverables

By the end of the pre-lab, you should have:

Experiment Log / Tuning Table

- Document each set of gains (e.g., motor PI gains, path-control gains, trapezoidal profile parameters) that you test on the real robot.
- Note the resulting behavior: overshoot, final position error, time to settle, or cross-track error.

Plots / Trajectory Visualizations

- At least one plot showing the **actual path** vs. the **desired path** on the real robot.
- Overlay the **simulated** vs. **real-world** trajectories in the same coordinate system to highlight discrepancies.

Comparison & Analysis

- A short **written comparison** between simulation results (from the Pre-Lab) and observed real-robot performance.
- Identify key reasons for any deviations (e.g., wheel slip, sensor noise, unit conversion issues).

Final Controller Settings

- Provide your **final, tuned gains** (e.g., Kp,Ki, path-control θ gain, cross-track error gain) that yielded acceptable performance.
- Include brief justifications for these parameter choices (why each gain was increased/decreased).

Optional Sensor Fusion Results

- If you implemented gyro-odometry fusion, show **before vs. after** performance metrics or plots.
- Note any reduction in heading drift or improvement in path accuracy.

Short Reflection or Lab Report

- Summarize the most important lessons learned about how real-world factors (slip, noise, friction) influence path control.
- Outline potential future improvements (e.g., better sensor calibration, advanced filters).

Conclusion

By completing this In-Lab Activity, you will gain practical experience implementing path control on an actual mobile robot. Comparing simulation to reality will illuminate the nuances of real sensor data, motor dynamics, and environmental factors. Through systematic tuning and data analysis, you'll refine your understanding of core control principles and better appreciate the complexities of robotics in a non-ideal world.

Appendix A:

Matlab script to convert matlab points to C++ style

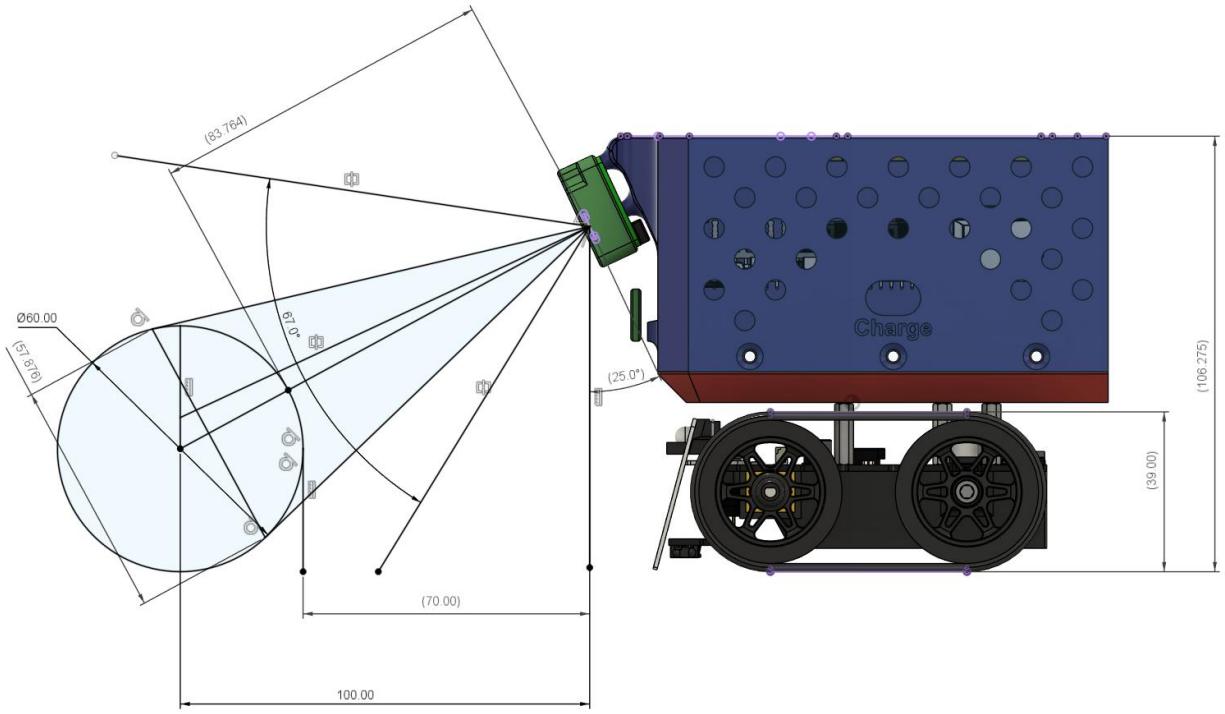
```
%% convert Car coordinates to c style
CarVec = [0 , 0 ; 1 , 1 ; 2 , 2]; % desired path in matlab style
output_str ='float desired_path[][2] = { ';
output_str = [output_str, '{', num2str(CarVec(1, 1)), ' , ', num2str(CarVec(1, 2)), '}'];
```

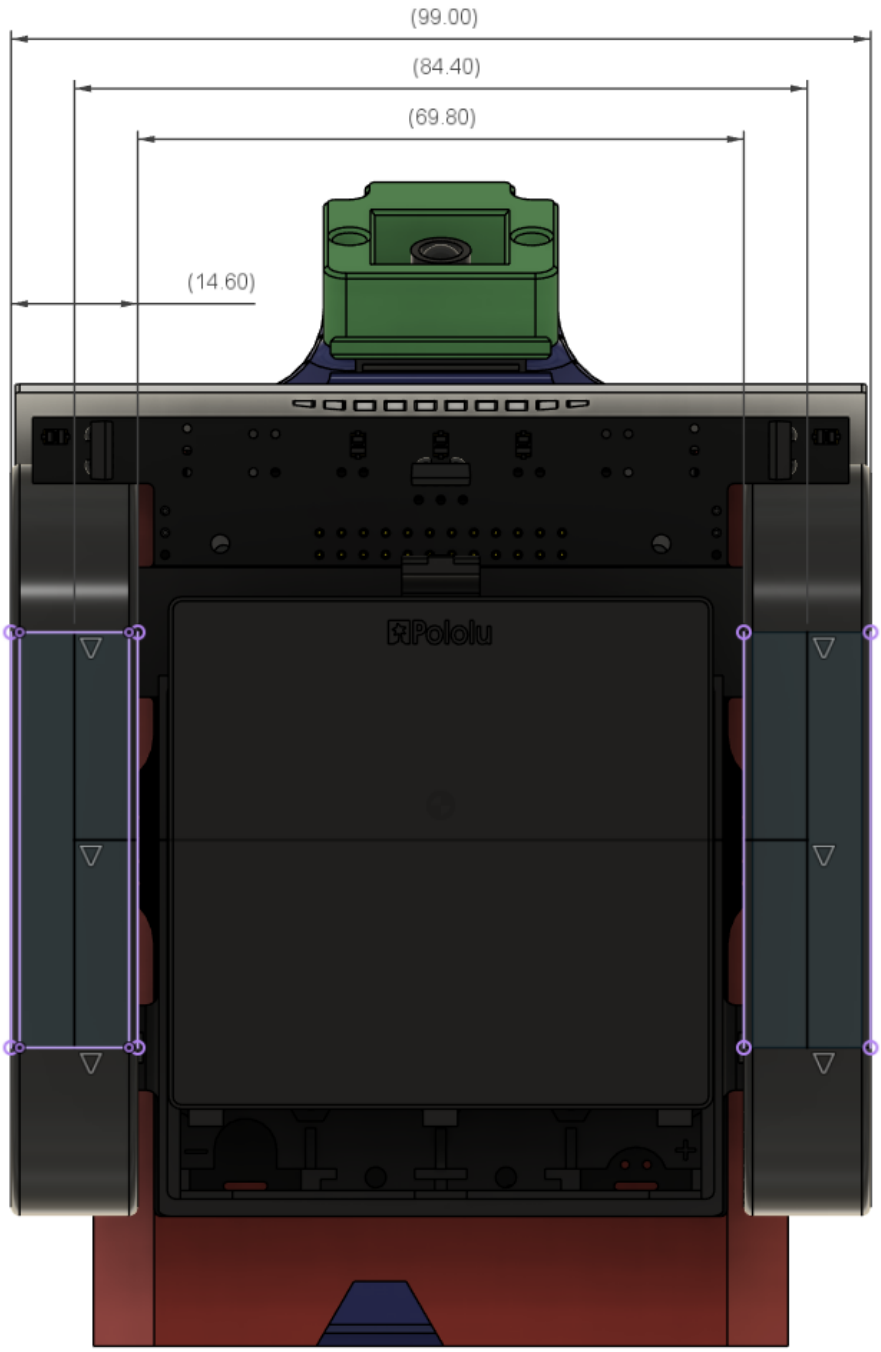
Python version (zumo_sim.paths.to_cpp), including the conversion from meters to millimetres used by the robot code:

```
from zumo_sim import paths
car_vec = [[0.1, 0], [0.2, 0.1], [0.3, 0.1]]       # desired path in meters
print(paths.to_cpp(car_vec, scale=1000))
# -> float desiredPath[][2] = { {100, 0}, {200, 100}, {300, 100} };
```

```
for i = 2:size(CarVec, 1)
% Append each row to the output as a C++ style array element
output_str = [output_str, ' , {', num2str(CarVec(i, 1)), ',', num2str(CarVec(i, 2)), '}'];
end
output_str = output_str + " };";
output_str
```

ZumoPi platform dimensions




(99.00)
(84.40)
(69.80)
(14.60)
Pololu